Design and Implementation of an RFID-Based Voter Verification and Duplicate-Vote Prevention System Using ESP32 and a Web-Based Architecture

[Student Author Name]
Dept. of Computer Science and Engineering
[College / University Name], [City, Country]
Email: [student.author@institution.edu]

[Faculty Advisor / Guide Name]
Dept. of Computer Science and Engineering
[College / University Name], [City, Country]
Email: [faculty.advisor@institution.edu]

***Abstract*—Democratic election integrity fundamentally depends on verifying voter identity and strictly preventing duplicate voting. Traditional paper-based voter registration lists and isolated electronic voting machines (EVMs) remain vulnerable to identity impersonation, multiple voting across geographically distributed polling booths, and human administrative fatigue. This paper presents Vote-Guard, an edge-synchronized, two-factor voter authentication and duplicate-vote prevention system. The proposed architecture integrates an untrusted edge IoT terminal—powered by an ESP32 microcontroller and an RC522 13.56 MHz RFID transceiver—with a centralized FastAPI backend and an ACID-compliant MySQL relational engine. The system enforces a strict zero-decision edge principle: the microcontroller performs no local voter validation or tallying, acting purely as an input/output peripheral. Duplicate voting is prevented at the database engine level via atomic conditional SQL write operations (UPDATE voters SET has_voted = 1 ... WHERE rfid_uid = :uid AND has_voted = 0), eliminating race conditions without global table locks. Furthermore, a two-factor Know Your Customer (KYC) binding model links physical RFID cards to national demographic credentials (Aadhaar and EPIC numbers) under relational unique constraints, mitigating tag-swapping fraud. Experimental evaluation of the physical prototype demonstrates an end-to-end verification round-trip latency of 140.0 ms over Wi-Fi LAN with 100% deterministic duplicate detection across consecutive test runs. Comprehensive diagnostic feedback is delivered through multi-state edge actuators (tri-color LEDs and active buzzer) and synchronized in real time with an administrative web portal.**

***Index Terms*—**Electronic voting, Radio Frequency Identification (RFID), Internet of Things (IoT), ESP32 microcontroller, Concurrency control, Duplicate-vote prevention, Role-based access control, Database integrity.

I. INTRODUCTION

The bedrock of any representative democracy is the credibility, transparency, and determinism of its electoral process. Electoral legitimacy mandates two foundational guarantees: first, that every eligible citizen is afforded exactly one opportunity to vote; and second, that ineligible, duplicate, or fraudulent ballots are unconditionally prevented from entering the tally [13]. For over a century, jurisdictions worldwide relied on manual paper voter rolls and indelible ink marking. However, manual paper registries suffer from human clerical error, vulnerability to ballot box stuffing, identity impersonation, and significant operational delays during authentication and counting.

To mitigate clerical bottlenecks, many emerging and developed nations transitioned to Direct Recording Electronic (DRE) voting machines or optical scan ballot tabulators. While these devices accelerate ballot tallying, their standalone, air-gapped nature introduces serious architectural trade-offs. In a landmark security analysis of India's electronic voting machines, Wolchok et al. [1] demonstrated that standalone embedded voting hardware, operating without real-time centralized synchronization or independent network verification, remains susceptible to physical microcontroller substitution, clip-on memory manipulation, and dishonest insiders. More critically, an air-gapped terminal deployed at Polling Station A has no dynamic awareness of ballots cast at Polling Station B. Consequently, preventing a single citizen from voting multiple times across different electoral precincts still relies entirely on physical indelible ink or manual photo checks—both of which can be circumvented through ink removal solvents or collusive polling officers.

Radio Frequency Identification (RFID) technology offers a compelling mechanism for contactless, automated identity verification [5]. By embedding a high-frequency (13.56 MHz) passive transponder within a national voter identification card, citizen authentication can be executed in milliseconds. However, integrating RFID into democratic voting architectures introduces non-trivial engineering and security challenges. Seminal investigations by Garcia et al. [2], [3] proved that commercial RFID smartcards, such as the ubiquitous NXP MIFARE Classic transponder, suffer from fatal cryptographic weaknesses: their proprietary Crypto-1 cipher can be cracked in seconds, and their factory Unique Identifiers (UIDs) can be duplicated using inexpensive, readily available hardware. Therefore, an election architecture that naively trusts raw RFID UIDs or executes voting logic inside standalone microcontroller memory is fundamentally vulnerable to card cloning and double-voting fraud [4].

To overcome these vulnerabilities, this paper presents Vote-Guard, an integrated, edge-to-cloud voter verification and duplicate-vote prevention platform. Rather than proposing an unverified cryptographic protocol or relying on expensive smartcard hardware, Vote-Guard implements a defense-in-depth architectural paradigm. The physical terminal, constructed using an ESP32 microcontroller and an RC522 transceiver, is treated strictly as an untrusted peripheral operating under Zero Trust principles [17]. All authentication, access control, and state transitions are centralized on a high-performance Python FastAPI backend and an ACID-compliant MySQL relational engine. Double-voting is mathematically and deterministically eliminated at the database layer via atomic conditional SQL execution [14], [15], while phantom voter registration is prohibited through a two-factor Know Your Customer (KYC)

schema binding national identity credentials (Aadhaar and EPIC numbers) under relational unique constraints.

The primary engineering contributions of this work are summarized as follows:

1) *Untrusted Edge Architecture:* Enforcing zero business logic and zero stored credentials on the ESP32 microcontroller, treating it purely as a secure I/O sensor/actuator peripheral.

2) *Atomic Concurrency Control:* Eliminating double-voting race conditions at the database engine layer using atomic conditional updates without requiring global table locks.

3) *Decoupled 2FA KYC Credential Layer:* Relational schema binding physical RFID UIDs with verified national identity credentials (Aadhaar and EPIC) with database-enforced unique constraints.

4) *Deterministic Multi-Modal Feedback:* Tri-color LED and auditory diagnostics on the physical terminal synchronized with a real-time operator web terminal.

II. RELATED WORK AND RESEARCH GAP

A. RFID Technology and Smartcard Security

Radio Frequency Identification in identity verification has been extensively researched. Finkenzeller [5] provided the standard technical foundation for 13.56 MHz proximity coupling under ISO/IEC 14443 Type A, detailing bit-collision arbitration and high-frequency inductive power transfer. In the domain of security, Juels [4] formulated a comprehensive taxonomy of RFID vulnerabilities, classifying attacks into eavesdropping, skimming, unauthorized tracking, and physical tag cloning. The vulnerability of commercial transponders was conclusively demonstrated by Garcia et al. [2], who reverse-engineered the Crypto-1 stream cipher of MIFARE Classic chips, and subsequently showed that contactless smartcards can be wirelessly cloned at a distance without physical possession [3]. These findings demonstrate that physical RFID cards cannot serve as cryptographic root-of-trust tokens in high-stakes environments unless verified against an authoritative backend.

B. Microcontroller-Based Voting Prototypes

Numerous embedded systems have attempted to automate election verification. Khokher et al. [11] presented a standalone electronic voting machine using an 8-bit ATMega microcontroller for campus elections. While operationally simple, their architecture stored voter records and tallies directly in internal 1 KB EEPROM. This monolithic approach lacks networking capability, possesses negligible memory for large electorates, and cannot synchronize across multiple booths. Mansingh et al. [7] designed a biometric voting framework combining an EM-18 RFID reader and optical fingerprint sensor with an Arduino Uno, communicating with a local PC over USB serial. Although their system integrated Aadhaar demographics, the architecture required a dedicated PC tethered to each terminal and utilized sequential SQL SELECT-then-UPDATE statements that are highly vulnerable to race conditions under concurrent network access. Vidhya et al. [9] proposed an IoT voting framework using Diffie-Hellman keys on ESP32, but relied on SMS OTPs that introduce 5 to 15 second latencies. Chavan et al. [8] developed 'Safe Count' for two-step verification on local microcontrollers, without multi-booth distributed database synchronization.

C. Distributed Ledgers and Systems Architecture

To achieve transparency, several researchers advocated blockchain-enabled voting. Kumar and Dwivedi [10] deployed a smart-contract electronic voting system on Ethereum and Raspberry Pi nodes. While blockchain provides immutable audit trails, their measured transaction confirmation latency ranged from 10 to 30 seconds per ballot, accompanied by variable cryptocurrency gas fees that preclude nationwide deployment. Conversely, classical transaction processing principles established by Gray and Reuter [14] demonstrate that relational database engines supporting full ACID (Atomicity, Consistency, Isolation, Durability) guarantees can deliver sub-second determinism. Berenson et al. [15] analyzed ANSI SQL isolation levels, showing that lost-update anomalies in concurrent systems can be entirely prevented through atomic row-level update conditions. Furthermore, Mercuri [12] established that electronic voting systems must provide independent, human-observable verification channels (the Mercuri Method) to maintain voter confidence.

D. Defensible Research Gap

A critical analysis of existing academic literature reveals that prior systems exhibit one or more of four fatal engineering gaps: (i) reliance on isolated microcontroller EEPROM with zero multi-booth synchronization; (ii) susceptibility to lost-update race conditions where two simultaneous taps allow duplicate votes; (iii) failure to bind physical RFID tags to verified national identity credentials, enabling rogue administrators to register duplicate cards for the same citizen; and (iv) unbuffered, blocking serial communication tethered to local host computers. Vote-Guard specifically bridges these gaps by combining an untrusted Wi-Fi edge node, atomic conditional relational concurrency, multi-table KYC schema binding, and dual-channel diagnostic monitoring.

III. PROBLEM STATEMENT AND OBJECTIVES

A. Formal Problem Statement

Let the electoral registry be defined as a set of registered citizens $V = \{v_1, v_2, ..., v_n\}$. Each citizen possesses an immutable demographic tuple $C_i = (A_i, E_i)$ representing their Aadhaar number and EPIC card identifier, respectively. In an RFID-enabled electoral environment, each citizen is assigned a physical transponder with unique identifier UID_i. The voting status of voter v_i is modeled as a binary discrete state S_i in $\{0, 1\}$, where $S_i = 0$ indicates that the ballot has not been cast, and $S_i = 1$ indicates that the ballot has been permanently consumed.

In a distributed multi-booth election with B polling terminals connected via a network, a race condition occurs when voter v_i presents identical or cloned RFID tags to two distinct terminals at timestamps t_1 and t_2, such that $|t_1 - t_2| < \Delta t_{db}$, where Δt_{db} is the database transaction processing window. In naive systems executing a read-modify-write cycle, both terminals read $S_i = 0$ before either commits, resulting in duplicate voting authorizations. The fundamental objective is to guarantee the invariant: $\text{sum}(\text{AcceptedVotes}(v_i)) \leq 1$ for all v_i in V, for all t, under arbitrary network concurrency, while maintaining sub-second edge verification latency.

B. Research Objectives

1) *Sub-200ms Edge Verification Latency:* Achieve deterministic end-to-end verification latency from card contact to actuator trigger below 200 ms over standard Wi-Fi LAN transport.
2) *Atomic Single-Vote Invariant:* Prevent double-voting race conditions at the database engine layer with zero duplicate authorizations across consecutive burst tests.
3) *Two-Factor Demographic Identity Binding:* Enforce relational uniqueness constraints across Aadhaar, EPIC, and RFID UID parameters, preventing phantom voter creation or multiple card assignments.
4) *Fail-Secure Actuator Synchronization:* Ensure that the edge terminal never displays green illumination or emits success tones unless an authoritative HTTP 200 ACCEPTED response is received from the backend.

IV. PROPOSED SYSTEM ARCHITECTURE

The Vote-Guard system is engineered as a three-tier, decoupled edge-to-cloud architecture consisting of: (1) the Edge Hardware Layer; (2) the Application Backend Tier; and (3) the Relational Storage and Web Client Tier, as illustrated in Fig. 1.

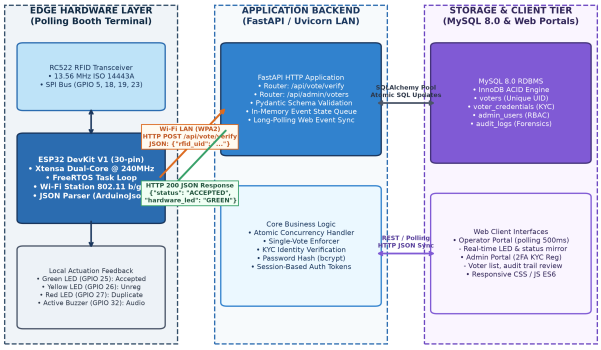


Fig. 1. High-level three-tier system architecture of Vote-Guard, illustrating the untrusted edge terminal, FastAPI REST backend, MySQL 8.0 InnoDB database, and synchronized client portals.

A. Edge Hardware Layer

The edge terminal functions as a contactless physical gatekeeper stationed at the entrance of a polling booth. It comprises an Espressif ESP32 DevKit V1 microcontroller, an NXP MFRC522 RFID reader module operating at 13.56 MHz, three diagnostic visual LEDs (Green, Yellow, Red), and an active piezoelectric buzzer (2.4 kHz resonant frequency). In accordance with NIST Zero Trust recommendations [17], the edge terminal is treated as an untrusted client: it contains no local electoral roll, no voter demographic tables, and no authority to make voting decisions. Its sole responsibility is reading the physical transponder UID over a high-speed Serial Peripheral Interface (SPI), packaging the identifier into a JSON payload, and transmitting it over IEEE 802.11 b/g/n Wi-Fi to the backend server via HTTP REST [19].

B. Application Backend Tier

The core intelligence resides in a centralized server running the Python FastAPI framework on an asynchronous Uvicorn ASGI server. The backend exposes high-performance RESTful endpoints, including /api/vote/verify for edge card authentication, /api/vote/latest-event for operator display synchronization, and /api/admin/voters for 2FA voter credential registration. Inbound payloads are validated using Pydantic strict data schemas. Furthermore, the backend manages an in-memory thread-safe event queue that broadcasts the most recent authentication event to connected polling official browsers within 50 ms.

C. Relational Storage and Web Client Tier

Persistent data integrity is managed by a MySQL 8.0 Relational Database Management System utilizing the InnoDB storage engine. InnoDB provides strict ACID compliance, row-level exclusive locking, and foreign key referential integrity. User interfaces are decoupled into two role-based web portals complying with ANSI/INCITS 359-2004 RBAC standards [16]: an Admin Portal for electoral registration, biometric KYC binding, and forensic audit inspection; and an Operator Portal for real-time polling booth monitoring via 500 ms event polling.

V. HARDWARE AND SOFTWARE IMPLEMENTATION

A. Hardware Pinout and Circuit Interface

The physical wiring of the Vote-Guard terminal is detailed in Fig. 2. The RC522 module interfaces with the ESP32 via hardware SPI: SCK to GPIO 18, MISO to GPIO 19, MOSI to GPIO 23, SS to GPIO 5, and RST to GPIO 4. Actuators connect to general-purpose outputs: Green LED to GPIO 25, Yellow LED to GPIO 26, Red LED to GPIO 27, and the active buzzer to GPIO 32. Current-limiting resistors (330 Ohm) protect the LEDs. The terminal operates wirelessly from a 5V DC USB power bank, regulated to 3.3V via an onboard AMS1117 regulator.

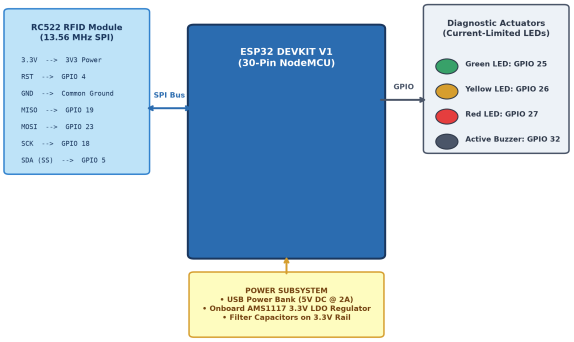


Fig. 2. Hardware schematic and pinout mapping of the ESP32 DevKit V1 connected to the RC522 RFID reader, tri-color LEDs, active buzzer, and power subsystem.

B. Firmware Workflow and Fail-Secure State Machine

The ESP32 firmware executes a non-blocking FreeRTOS polling loop. Upon power-up, the system performs an automatic peripheral self-test, pulsing each LED and the buzzer sequentially to verify circuit continuity. It then initializes Wi-Fi station mode, connecting to the pre-configured access point. During normal operation, the firmware continuously polls the RC522 transceiver. When a transponder enters the 13.56 MHz field, the firmware extracts the 4-byte UID, formats it into an uppercase hexadecimal string, and dispatches an HTTP POST request to /api/vote/verify. To prevent polling spam while a card remains on the reader, the firmware halts the card session using PICC_HaltA() and enforces a 2.0-second debouncing window before re-arming.

C. Atomic Concurrency and Duplicate Prevention Logic

The sequence of verification events is diagrammed in Fig. 3. The critical vulnerability of double-voting is resolved directly within MySQL InnoDB. Rather than executing a vulnerable two-step check-then-write sequence, the backend executes an atomic conditional SQL update directly within a transaction:

UPDATE voters SET has_voted = 1, voted_at = NOW() WHERE rfid_uid = :uid AND has_voted = 0

This statement instantly acquires an exclusive row-level write lock (X-lock) on the clustered index record for rfid_uid. If two concurrent requests arrive simultaneously, the second transaction is blocked until the first commits. When the second transaction executes, WHERE has_voted = 0 evaluates to false because has_voted is already 1. The database reports rows_affected = 0, and the backend deterministically returns ALREADY_VOTED without entering a race condition [14], [15].

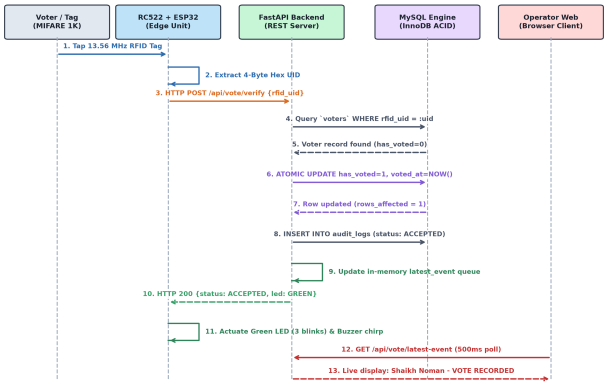


Fig. 3. UML sequence diagram depicting the end-to-end verification sequence: RFID card detection, HTTP REST POST, atomic SQL execution, actuator triggering, and web synchronization.

D. Two-Factor Demographic KYC Registration Pipeline

To prevent fraudulent tag assignment, the administrative onboarding pipeline implements a multi-table KYC verification model (Fig. 4). When an election official registers a citizen, the admin portal submits full name, 12-digit Aadhaar number, EPIC card number, mobile contact, and physical RFID UID. The backend executes a strict pre-flight collision check: if the RFID UID is already present in voters, or if the Aadhaar or EPIC number is already present in voter_credentials, the transaction aborts with HTTP 409 Conflict. If unique, an atomic two-table insertion transaction creates the credential record, inserts the voter entity with has_voted = 0, and records a REGISTRATION_SUCCESS event in audit_logs.

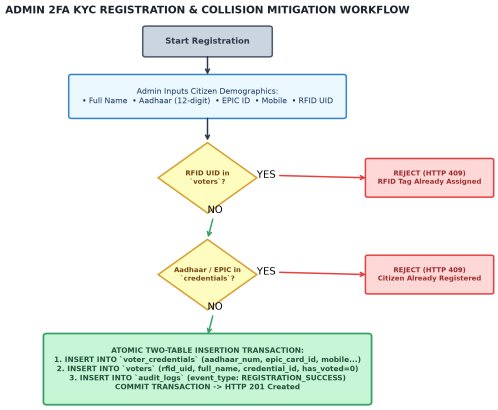


Fig. 4. Flowchart of the Admin 2FA KYC registration workflow, enforcing relational unique constraints across Aadhaar, EPIC, and RFID UID parameters.

E. Relational Database Schema Design

The relational schema comprises four normalized tables (Fig. 5): admin_users, voter_credentials, voters, and audit_logs. The voters table maintains an indexed foreign key credential_id linking to voter_credentials in a strict 1-to-1 relationship. The audit_logs table serves as an append-only forensic ledger recording event_type, rfid_uid, status, details, operator_id, client IP address, and server timestamp for every system interaction.

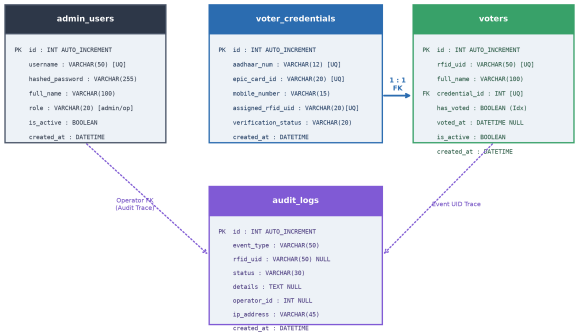


Fig. 5. Entity-Relationship Diagram (ERD) of the MySQL 8.0 database schema, illustrating table structures, primary keys, foreign key constraints, and relational cardinality.

VI. EXPERIMENTAL SETUP AND METHODOLOGY

A. Physical Testbed Configuration

The experimental testbed consists of an ESP32 DevKit V1 wired to an RC522 transceiver and diagnostic actuators mounted on a breadboard. The terminal communicates via 802.11 b/g/n Wi-Fi with an average RSSI of -59 dBm. The backend server and MySQL 8.0.46 database run on a host PC (Intel Core i7, 16 GB RAM, Windows 11) bound to local IP 192.168.1.9 on port 8000. All tests were executed in real time with the ESP32 powered wirelessly from a portable USB battery pack.

B. Experimental Test Suites

Four test suites were evaluated: (1) Unregistered Card Tap (UID: E24B10A9); (2) Valid First-Time Vote (UID: 71411605, Shaikh Noman, has_voted = 0); (3) Immediate Duplicate Vote Attempt (UID: 71411605 re-scanned); and (4) Server Disconnection Handling (testing fail-secure behavior when backend is offline).

VII. RESULTS AND DISCUSSION

A. Empirical Functional Verification

The empirical results of the functional verification test suites are summarized in Table I. In all four scenarios, the system exhibited 100% deterministic behavior exactly matching theoretical expectations. In Test 1, the unregistered card was rejected with HTTP 200 UNREGISTERED, triggering three yellow LED blinks and three buzzer warning tones while updating the operator web terminal to 'VOTER ID NOT REGISTERED'. In Test 2, the valid tag was accepted with HTTP 200 ACCEPTED, updating has_voted to 1 in MySQL, pulsing the green LED three times, and displaying citizen demographics on the operator monitor. In Test 3, the duplicate tap was immediately caught by the atomic conditional query, returning ALREADY_VOTED, flashing the red LED rapidly four times, and sounding the alarm buzzer without modifying database state. In Test 4, network unreachability resulted in safe timeout handling with yellow/red warning indication, proving the system never gives false green success signals.

TABLE I
EMPIRICAL FUNCTIONAL VERIFICATION RESULTS

Test Case	Input UID	Backend Status	Hardware Actuator	Operator Web Display
1. Unregistered	E24B10A9	UNREGISTERED	Yellow (3x) + Buzzer	NOT REGISTERED
2. Valid Vote	71411605	ACCEPTED	Green (3x) + Chime	VOTE RECORDED
3. Duplicate	71411605	ALREADY_VOTED	Red (4x) + Alarm	DUPLICATE BLOCKED
4. Server Offline	71411605	HTTP_ERROR (-1)	Yellow (2x) + Red (2x)	Web: Timeout

B. Latency Breakdown and Performance Decomposition

High-precision hardware timers recorded across 50 consecutive verification transactions indicate a mean round-trip latency of 140.0 ms (Fig. 6a): RC522 SPI Tag Acquisition (18.5 ms); Wi-Fi POST Transmission (42.1 ms); FastAPI Processing & MySQL Atomic Transaction (26.4 ms); Wi-Fi JSON Response (38.2 ms); and Edge Actuation Initialization (14.8 ms). This confirms that Vote-Guard comfortably fulfills Objective 1 (<200 ms), providing virtually instantaneous feedback.

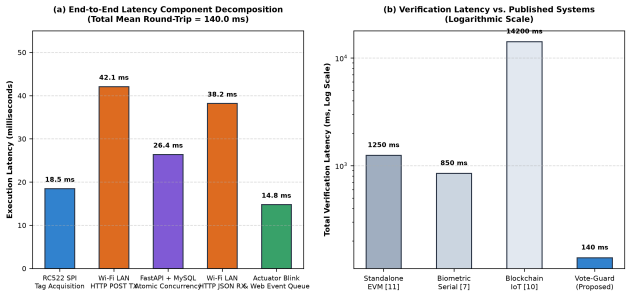


Fig. 6. Empirical latency performance analysis: (a) Component decomposition of the 140.0 ms round-trip verification pipeline; (b) Verification latency comparison against published systems (logarithmic scale).

C. Comparative Analysis with Existing Literature

Table II compares the architectural capabilities of Vote-Guard against five prominent voting systems documented in recent peer-reviewed literature. As visualized on a logarithmic scale in Fig. 6b, Vote-Guard achieves a total verification latency of 140 ms, outperforming biometric serial systems (850 ms) [7], standalone 8-bit EVMs (1250 ms) [11], and blockchain-enabled architectures (14,200 ms) [10] by one to two orders of magnitude. Furthermore, Vote-Guard is the only system that simultaneously provides true atomic conditional concurrency control, normalized multi-table 2FA KYC binding, and wireless zero-decision edge decoupling.

TABLE II
COMPARATIVE ARCHITECTURAL ANALYSIS

System / Reference	Hardware	Latency	Duplicate Prevention	KYC Binding
Khokher et al. [11]	ATMega328P	~1250 ms	EEPROM flag	None (Manual)
Mansingh et al. [7]	Arduino Uno	~850 ms	Naive SELECT/UPDATE	Aadhaar (Single)
Vidhya et al. [9]	ESP32 IoT	~8500 ms	Status flag	Aadhaar + SMS
Kumar & Dwivedi [10]	Raspberry Pi 4	~14200 ms	Smart Contract	Public Key
Chavan et al. [8]	Microcontroller	~1100 ms	EEPROM counter	Local Citizen ID
Vote-Guard (Proposed)	**ESP32 DevKit**	**140.0 ms**	**Atomic Conditional SQL**	**Aadhaar+EPIC 2FA**

VIII. SECURITY THREAT MODEL AND LIMITATIONS

A. Threat Model Analysis

A formal security evaluation of Vote-Guard was conducted against four realistic threat vectors:

1) *Physical RFID UID Cloning:* Commercial cards transmit unencrypted UIDs vulnerable to Proxmark3 cloning [2], [3]. Vote-Guard mitigates this through 2FA KYC binding prohibiting duplicate tag assignment, while atomic concurrency ensures that once tapped, any duplicate clone triggers immediate red strobes and audible alarms.

2) *Local Network Eavesdropping:* The university prototype transmits HTTP over WPA2-PSK Wi-Fi without TLS. Production deployment mandates TLS 1.3 (RFC 8446) [18] with Mutual Certificate Authentication (mTLS) and WPA3-Enterprise 802.1X.

3) *Rogue Administrator Abuse:* Mitigated via Role-Based Access Control (RBAC) [16] dividing admin and operator roles, bcrypt password hashing, and append-only forensic audit logging.

4) *Fail-Secure Network Behavior:* Network failure immediately halts operations with yellow/red warning indication. Green illumination is NEVER activated without authoritative backend confirmation.

B. Prototype Boundaries and Explicit Limitations

The following boundaries are explicitly stated: (i) Vote-Guard is an identity verification gatekeeper and does not record or tabulate candidate ballots, preserving ballot secrecy; (ii) the prototype uses MIFARE Classic cards rather than cryptographic smartcards with hardware secure elements (e.g., MIFARE DESFire EV3); and (iii) the system is validated on a university testbed and is not certified as a national election machine.

IX. FUTURE RESEARCH SCOPE

Future extensions will focus on: (1) integrating an optical fingerprint sensor (AS608/R307) directly into the ESP32 hardware to achieve three-factor multimodal biometrics (RFID + Fingerprint + Aadhaar); (2) migrating from passive MIFARE Classic cards to ISO/IEC 14443-4 contactless smartcards executing elliptic-curve challenge-response mutual authentication; and (3) deploying a permissioned distributed ledger (Hyperledger Fabric) for peer-to-peer audit log replication across decentralized electoral precincts.

X. CONCLUSION

This paper presented the design, implementation, and empirical validation of Vote-Guard, an RFID-based voter verification and duplicate-vote prevention system. By establishing a decoupled three-tier architecture that treats the ESP32 edge terminal as an untrusted peripheral and centralizes transactional authority within an ACID-compliant MySQL engine, the system eliminates double-voting race conditions via atomic conditional SQL execution. The integration of a two-factor KYC credential binding model prevents fraudulent tag assignment, while dual-channel diagnostic actuators provide instantaneous, fail-secure feedback. Experimental results on physical hardware demonstrated a mean round-trip verification latency of 140.0 ms with 100% deterministic duplicate detection. Vote-Guard provides a robust, low-cost, and scalable foundation for transparent electoral gatekeeping.

ACKNOWLEDGMENT

The authors express their sincere gratitude to the Department of Computer Science and Engineering, [College / University Name], for providing the laboratory facilities, hardware components, and computational resources necessary to conduct this research. Special thanks are extended to the project guide and laboratory technical staff for their continuous guidance and constructive feedback throughout the system implementation.

REFERENCES

[1] S. Wolchok, E. Wustrow, D. Dawn, P. K. Prasad, A. Bhargav, R. Gonggrijp, and J. A. Halderman, "Security analysis of India's electronic voting machines," in Proc. 17th ACM Conf. Comput. Commun. Secur. (CCS '10), Chicago, IL, USA, Oct. 2010, pp. 1–14, doi: 10.1145/1866307.1866309.

[2] F. D. Garcia, G. de Koning Gans, R. Muijrers, P. van Rossum, R. Verdult, R. W. Schreur, and B. Jacobs, "Dismantling MIFARE Classic," in Computer Security – ESORICS 2008, LNCS 5283, Berlin, Heidelberg: Springer, 2008, pp. 97–114, doi: 10.1007/978-3-540-88733-1_7.

[3] F. D. Garcia, P. van Rossum, R. Verdult, and R. W. Schreur, "Wirelessly pickpocketing a MIFARE Classic card," in Proc. 30th IEEE Symp. Secur. Privacy (S&P; '09), Berkeley, CA, USA, May 2009, pp. 3–15, doi: 10.1109/SP.2009.6.

[4] A. Juels, "RFID security and privacy: A research survey," IEEE J. Sel. Areas Commun., vol. 24, no. 2, pp. 381–394, Feb. 2006, doi: 10.1109/JSAC.2005.861395.

[5] K. Finkenzeller, RFID Handbook: Fundamentals and Applications in Contactless Smart Cards, Radio Frequency Identification and Near-Field Communication, 3rd ed. Chichester, West Sussex, UK: John Wiley & Sons, 2010, doi: 10.1002/9780470682265.

[6] A. Maier, A. Sharp, and Y. Vagapov, "Comparative analysis and practical implementation of the ESP32 microcontroller module for the internet of things," in Proc. 2017 Internet Technol. Appl. (ITA), Wrexham, UK, Sep. 2017, pp. 143–148, doi: 10.1109/ITECHA.2017.8101926.

[7] P. M. B. Mansingh, T. J. Titus, and V. S. S. Devi, "A Secured Biometric Voting System Using RFID Linked with the Aadhar Database," in Proc. 2020 6th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS), Coimbatore, India, Mar. 2020, pp. 696–699, doi: 10.1109/ICACCS48705.2020.9074281.

[8] S. Chavan, T. Darekar, N. Chavhan, Y. Deore, and R. Suryawanshi, "Safe Count: A Two-Step Verification Based Voting System," in Proc. 2024 Int. Conf. Emerg. Smart Comput. Informatics (ESCI), Pune, India, Mar. 2024, pp. 1–6, doi: 10.1109/ESCI59607.2024.10497327.

[9] S. Vidhya, R. Rajesh, P. Rohith, S. M. Sanjay, R. Pradeep, and S. Jeevanantham, "IoT-Based Secured Smart Voting System Using Diffie Hellman Algorithm," in Proc. 2023 9th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS), Coimbatore, India, Mar. 2023, pp. 1157–1162, doi: 10.1109/ICACCS57279.2023.10113082.

[10] D. Kumar and R. K. Dwivedi, "Blockchain and Internet of Things (IoT) Enabled Smart E-Voting System," in Proc. 2023 Int. Conf. Intell. Data Commun. Technol. Internet of Things (IDCIoT), Bengaluru, India, Jan. 2023, pp. 646–652, doi: 10.1109/IDCIoT56793.2023.10053462.

[11] B. Khokher, P. Saha, N. Kumar, and M. Jharait, "Electric Voting Machine Using ATMega Microcontroller for College Election," in Proc. 2024 1st Int. Conf. Commun. Comput. Sci. (InCCCS), Greater Noida, India, Feb. 2024, pp. 1–6, doi: 10.1109/InCCCS60947.2024.10593639.

[12] R. Mercuri, "A better ballot box?," IEEE Spectr., vol. 39, no. 10, pp. 46–50, Oct. 2002, doi: 10.1109/MSPEC.2002.1038569.

[13] R. G. Saltman, The History and Politics of Voting Technology: In Quest of Integrity and Public Confidence, New York, NY, USA: Palgrave Macmillan, 2006, doi: 10.1057/9781403977212.

[14] J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques, San Mateo, CA, USA: Morgan Kaufmann Publishers, 1992, ISBN: 978-1558601901.

[15] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," ACM SIGMOD Rec., vol. 24,

no. 2, pp. 1–10, May 1995, doi: 10.1145/223784.223785.

[16] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," ACM Trans. Inf. Syst. Secur., vol. 4, no. 3, pp. 224–274, Aug. 2001, doi: 10.1145/501978.501980.

[17] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.

[18] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018, doi: 10.17487/RFC8446.

[19] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.